

Spring Microservice Architecture Documentation

1. Core idea

Three internal microservices (auth, user, payment), one API gateway, and a separate admin project that lives outside the cluster and talks to it over HTTP

There are two distinct trust models:

- Admin routes are public (no token). They are secured solely by checking the caller's network origin - host / IP / domain / port – against an allowlist. If the request comes from an allowed admin origin, it is treated as admin; if not, it is rejected. The admin project never logs in and never carries a JWT.
- User routes use identity (JWT). Users log in through the auth service and carry a `ROLE_USER` token. The service resolves the user id and roles directly from the token.

A per-service path config ties it together: each service has a list marking which paths are admin routes (origin-checked, public) versus user routes (token-checked).

Worth knowing: relying only on network origin for admin means anyone who can originate traffic from an allowed host/IP can perform admin actions, so the admin project should sit on a private network. Section 11 lists optional hardening if you ever want a second factor.

2. Services and responsibilities

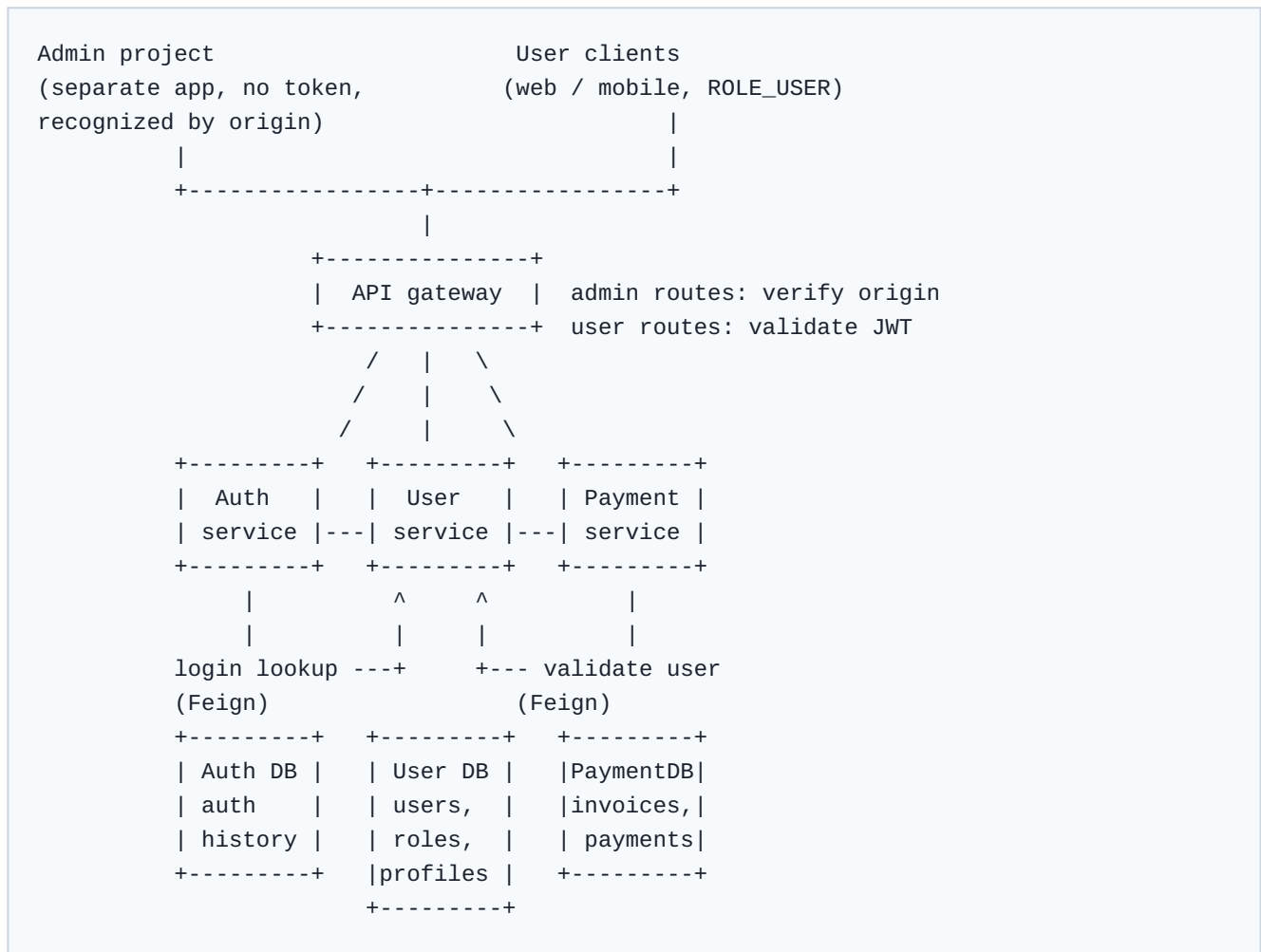
API gateway –single entry point. For user routes it validates the JWT and passes it through unchanged. For admin routes it skips token checks and instead verifies the source host / IP / domain / port against the admin allowlist, rejecting anything that doesn't match. Each service resolves the user identity from the JWT itself.

Auth service–owns authentication history only, not user records, It handles user login, token issuance, refresh, and logout, and records every authentication event. On login it sends a user-inquiry request to the user service to fetch the account (credential hash + roles), verifies the password, then issues the JWT. The admin project does not use this service,

User service – owns the user records, the roles (roles live on the user, in the user database), and the profile/information fields. It exposes an internal inquiry endpoint used by the auth service, plus profile read/update endpoints. It is also queried by the payment service to validate a user when an invoice is created.

Payment service – owns invoices and payments, Invoice creation, listing, and cancellation are admin routes (origin-checked). Paying and listing one's own invoices are user routes (token-checked). It calls the user service to confirm the target user exists when an invoice is created.

3. Component overview



Two service-to-service calls both target the user service: auth → user (login lookup) and payment → user (invoice validation).

Optional supporting infrastructure: a service registry (Eureka) for service-name resolution and a config server for centralized config.

4. Header propagation strategy

A small set of standard headers flows through every hop. Admin requests carry no token; they are recognized only by verified origin. On user routes each service resolves the user id and roles directly from the JWT. When one service calls another, the Authorization token (user routes) and X-Request-Id are forwarded so identity and trace context survive the second hop.

Header	Set by	Meaning
Authorization	client	Bearer <jwt> (present on user routes only)
X-Request-Id	gateway	correlation / trace id

5. Admin recognition (per-service path config)

Each service keeps a config list marking which paths are admin routes. The decision works like this:

1. The gateway checks whether the requested path is an admin route.
2. If it is, the gateway verifies the caller's source host / IP / domain / port against the admin allowlist. A match is forwarded; a mismatch is rejected. No token is involved
3. If the path is a user route, the gateway validates the JWT and passes it through; the service resolves the user id and roles from the token.

A user who hits an admin route is rejected at the origin check, because their hostname / IP will not be on the admin allowlist. So "is this admin?" is answered entirely by verified network origin, scoped to the paths the config marks as admin routes.

6. Service-to-service calls

When a service calls another, an interceptor forwards the standard headers from the incoming request – the Authorization token (on user-initiated chains) and X-Request-Id -so the downstream service can resolve the same identity and trace id. Admin-initiated chains carry no token, only X-Request-Id -

Two concrete calls in this design:

- Auth → User (login lookup): on user login, auth asks the user service for the account by username, receiving the credential hash and roles.
- Payment → User (invoice validation): when an admin creates an invoice, payment confirms the target user exists before writing the invoice.

Operational note: the request context the interceptor reads from is thread-bound. If a service makes these calls from an async task or a separate thread pool, the context must be propagated explicitly or the headers won't be forwarded

7. Database structure (database-per-service)

Each service owns its schema; nothing does cross-database joins. Cross-service references are stored as plain IDs, not foreign keys.

Auth DB – authentication history only:

- auth_history – id, user_id, username, event_type (LOGIN_SUCCESS /LOGIN_FAIL/ REFRESH/LOGOUT), ip_address, user_agent, created_at
- refresh_tokens (optional, if refresh tokens are tracked server-side) - id, user_id, token, expires_at, revoked

User DB –accounts, roles, and profile info:

- users – id, username (unique), email, password_hash, enabled, created_at, updated_at

- `user_roles` – `user_id`, `role` (roles belong to the user, stored here in the user service; e.g. `ROLE_USER`, `ROLE_ADMIN`)
- `user_profiles` - `id`, `user_id`, `full_name`, `phone`, `billing_address`, `tax_id`, `created_at`, `updated_at`

Payment DB – invoices and payments:

- `invoices` – `id`, `invoice_number` (unique), `user_id`, `amount`, `currency`, `status` (`UNPAID` / `PAID` / `CANCELLED`), `description`, `due_date`, `created_at`, `updated_at`
- `invoice_items` – `id`, `invoice_id`, `name`, `qty`, `unit price`, `line total`
- `payments` – `id`, `invoice_id`, `user_id`, `amount`, `method`, `provider_ref`, `status`, `paid_at`

8. Endpoints

Access column: (admin origin) = public route, network-verified, no token; (USER) = JWT user route; (public) = open; (internal) = service-only

Auth service

- `POST /api/auth/login` – verify credentials via the user service, issue tokens, record the attempt. (public)
- `POST /api/auth/refresh` – rotate token, record event. (public)
- `POST /api/auth/logout` - revoke/ record event. (USER)
- `GET /api/auth/history/me` – own login history. (USER)
- `GET /api/auth/history` – all login history. (admin origin)

User service

- `POST /api/users` – create account (registration). (public)
- `GET /api/users/me` – own account + profile. (USER)
- `GET /api/users/{id}` – a user. (owner USER, or admin origin)
- `PUT /api/users/{id}` – update profile. (owner USER, or admin origin)
- `GET /api/users/internal/by-username/{username}` – credential + role lookup. (internal –auth service only)

Payment service

- `POST /api/payments/invoices` - create invoice, (admin origin)
- `GET /api/payments/invoices` - list all invoices. (admin origin)
- `GET /api/payments/invoices/mine` -own invoices. (USER)
- `GET /api/payments/invoices/{id}` – one invoice, (owner USER, or admin origin)
- `POST /api/payments/invoices/{id}/pay` – pay invoice. (USER, owner)
- `POST /api/payments/invoices/{id}/cancel` - cancel invoice. (admin origin)

9. Login / auth flow (users)

1. Client sends credentials to POST /api/auth/login through the gateway.
2. Auth service sends a user-inquiry request to the user service (by username), receiving the credential hash and roles.
3. Auth service verifies the password and issues a JWT whose role claims are the roles returned by the user service.
4. Auth service records the attempt (success or failure) in auth_history
5. On later user requests, the gateway validates the JWT and passes it through; each service resolves the user id and roles from the token's claims when needed.

The admin project skips all of this – its requests are recognized by origin, not by login. Because the password hash lives with the user record in the user service, the internal by-username lookup must be locked down so only the auth service can call it.

10. Admin plan and the invoice - payment flow

The admin is a standalone deployable with its own UI and backend. It goes through the gateway, and its requests are recognized purely by verified network origin – no login, no token.

Invoice creation (admin):

1. Admin calls POST /api/payments/invoices through the gateway.
2. The gateway sees this is an admin route, verifies the source host / IP / domain / port is on the admin allowlist, and forwards the request.
3. Payment validates the target user by calling the user service (headers forwarded), then writes the invoice as UNPAID.

Payment (user):

1. User logs in, calls GET /api/payments/invoices/mine to list bills.
2. User calls POST/api/payments/invoices/{id}/pay
3. Payment resolves the user id from the JWT token and checks ownership (invoice. user_id matches it, processes the payment, and flips the invoice to PAID.

A user request to an admin route is rejected because its hostname / IP differs from the admin origin and will not match the allowlist.

11. Notes for later

- Keep the admin project on a private network or behind a VPN; origin-only security is only as strong as that boundary.

- Optional second factor if you want to harden the admin path without adding a full login: mutual TLS between the admin project and the gateway, or a shared secret header validated alongside the origin check.
- Be careful reading the source IP behind load balancers trust forwarded-for values only from your own proxies, or the origin check can be spoofed.
- Add a service registry and config server for service-name resolution and centralized configuration.
- Use idempotency keys on the payment-provider call so a retried "pay" never double-charges.

12. Development plan

Sequenced by dependency: the user service is built first because both auth (login lookup) and payment (invoice validation) depend on it. Each phase has a clear "done" bar so you can integrate continuously rather than big-bang at the end

Suggested stack: Spring Boot, Spring Cloud Gateway, Spring Cloud OpenFeign, Spring Security (JWT), a migration tool (Flyway or Liquibase), Testcontainers for integration tests, and docker-compose for local infra. Service registry (Eureka) and config server are optional and can be added in Phase 0 or deferred

Phase 0 - Foundations & scaffolding

- Decide repo layout (multi-module vs polyrepo) and set up the build + CI.
- Create a shared library for cross-cutting concerns: standard header constants, the Feign header-forwarding interceptor, the request-id filter, and a common error/response model.
- Provision one database per service and a docker-compose for local dev.
- Stand up a gateway skeleton that only routes (no security yet) plus health checks on each service.
- Done when: all services boot, the gateway routes a health check to each, CI is green.

Phase 1 - User service (dependency root)

- Schema + migrations for users, user_roles, user_profiles
- Endpoints: register, get-me, get/update by id.
- Internal by-username lookup returning credential hash + roles, locked down so only the auth service can reach it.
- Done when: a user can be created and fetched, and the internal lookup returns the credential hash and roles.

Phase 2- Auth service

- Schema + migrations for auth_history (and refresh_tokens if used).
- Login that calls the user service over Feign, password verification, JWT issuance, refresh, and logout.
- Record every authentication event.

- Done when: a user can log in and receive a valid JWT and the event is recorded.

Phase 3 – Gateway security

- JWT validation and pass-through for user routes.
- Path config marking admin routes; origin allowlist check (host / IP / domain / port) for those routes.
- Correct forwarded-for handling so the origin check can't be spoofed behind a load balancer.
- Done when: user routes require a valid JWT, admin routes are only reachable from an allowlisted origin, and a spoofing attempt is rejected.

Phase 4 - Payment service

- Schema + migrations for invoices, invoice items, payments.
- Admin routes: create / list / cancel invoice, validating the target user via Feign to the user service.
- User routes: list mine, get one, pay – with ownership resolved from the JWT.
- Done when: an admin (from the admin origin) creates an invoice, a user pays it, and a user is blocked from creating one.

Phase 5–Admin project integration

- Wire the external admin app to call through the gateway from an allowlisted origin.
- Exercise the full path: admin creates an invoice, user pays it.
- Done when: the end-to-end happy path works across both projects and a user hitting an admin route is rejected by the origin check

Phase 6-Cross-cutting hardening & ops

- Confirm header propagation (Authorization + request-id) across every service-to-service call, including async paths.
- Centralized error handling and request validation.
- Observability: structured logging keyed by request-id, basic metrics, health and readiness checks.
- Idempotency on the payment-provider call.
- Security review: internal endpoint lockdown, origin check behind the LB, and the optional second factor (mTLS or shared-secret header) if wanted.
- Done when: traces correlate across services, retried payments are safe, and the security checks pass.

Phase 7- Testing & release

- Unit and slice tests; Feign consumer/provider contract tests; integration tests with Testcontainers; an end-to-end test of both flows.
- Pipeline: build, test, containerize, deploy.
- Done when: the pipeline is green and produces deployable artifacts.

Sequencing notes

- Critical path: Phase 0 → 1 - 2 → 3, then 4, then 5.
- Parallel work: once the user service contract (Phase 1) is fixed, payment (Phase 4) and gateway security (Phase 3) can be built alongside auth (Phase 2) by mocking the user service against that contract.
- Cross-cutting (Phase 6) runs continuously rather than as a single late phase – fold each item in as the relevant service lands.